

# Intro to MCP Servers

Presented by Pranav



x



sase

# Install now

SETUP · README.md

Windows

setup.ps1 -CoreOnly

Mac

bash setup.sh --core-only

Pair if blocked



QR: event check-in



# Today's workshop

---

8 min setup · 10 theory · 7 demo · 15 practice  
7 security · 5 enterprise · 3 exit · 5 spare

## UNDERSTAND

Connect an AI Application to useful software capabilities.

## TRACE

Follow a request from the host to the music Application.

## IMPLEMENT + TEST

Extend the supplied server with one validated tool.

# What MCP is

---

## Model Context Protocol

An open standard that connects AI Applications to tools, data, and reusable instructions.

An MCP server is the program that exposes those capabilities.



Today: extend an existing server using the Python SDK.

# Why MCP instead of one prompt?

| Workflow step     | What MCP adds                                      |
|-------------------|----------------------------------------------------|
| One-shot prompt   | Gives instructions; tools still need a connection. |
| Live capabilities | The host discovers tools and input schemas.        |
| Act + observe     | Call an action, read state, then adjust.           |
| Reusable workflow | Compatible hosts reuse the same tool interface.    |



**Me when the next compatible AI host uses the same server.**

**MCP does not guarantee correctness. Test tools and verify outcomes.**

# Host, client, server: the apps you know

## HOST · the AI Application you use

VS Code + GitHub Copilot  
Agent chat coordinates model, tools and permissions.

### CLIENT · the MCP connection inside it

Built into the VS Code host  
Discovers tools, sends calls, receives results.



MCP

## SERVER · exposes tools

Lab: `workshop/starter/mcp_server_sdk.py`  
Built with SDK FastMCP  
`get_state` + `set_tempo`; you add `set_swing`

Model ≠ host: the selected model reasons; VS Code coordinates the calls.  
Backend ≠ MCP server: `app.py` changes the music; the server exposes its controls.

# Where an API fits

---

## AI HOST

Contains the MCP client

## MCP SERVER

Exposes a useful tool

## EXISTING API

Applies Application rules

## DATA / ACTION

Reads or changes the app

MCP request →

API call →

Read / write →

An MCP tool can call an EXISTING API, Python function, or supported Application command.

1 / 3 · PRIZE QUESTION · Hands up!

Who chooses which MCP tool to call?

A The host   B The server   C JSON

# 12 AM. Your team's MCP power tools.

Connect existing servers. Spend your time on the idea.



**Figma**

DESIGN

Bring UI designs into your coding workflow.



**GitHub**

BUILD

Inspect code, track issues and work with PRs.



**Supabase**

DATA

Inspect schemas, query data and manage migrations.



**Playwright**

TEST

Drive the browser and verify the demo flow.



**Vercel**

SHIP

Manage deployments and inspect deployment logs.



**Sentry**

DEBUG

Investigate errors before the judges find them.



**Stripe**

PAYMENTS

Explore APIs and build a sandbox payment demo.



**Context7 · Upstash**

DOCS

Pull current library docs and code examples.

**Design → build → connect data → test → deploy → fix**

Pick a compatible host → add a server → authorize → inspect tools → use it

Start with the 2–3 servers your demo needs. Access and features depend on permissions and plans.

# Tools, resources, and prompts

---

## TOOLS

Callable operations  
Example: `set_tempo` or  
`get_state`



## RESOURCES

Read-only context and data  
Our app: GeneralUser GS  
instrument catalog



## PROMPTS

Reusable task instructions  
Music demo:  
`compose_music(description)`



# Discover → select → invoke

---

## 1 · DISCOVER

The client lists tools with their names, descriptions, and input schemas.

## 2 · SELECT

The model proposes a tool and arguments; the host coordinates the call.

## 3 · INVOKE

The client calls the tool. The server validates, executes, and returns a result.

`set_tempo` accepts `bpm`, an integer.  
Python validation enforces the allowed range.

# Ways to build or connect MCP

---

| Option             | When to choose it                                                   |
|--------------------|---------------------------------------------------------------------|
| SDK FASTMCP        | Official Python SDK v1; used in this workshop.                      |
| Standalone FastMCP | Separate Python package: <code>from fastmcp import FastMCP</code> . |
| Other SDKs         | Official SDKs for your stack, including TypeScript.                 |
| Zapier MCP         | Managed app actions via Zapier MCP; a Zap is trigger → actions.     |

**DATABUTTON** · adjacent app builder

An app builder is not itself an MCP framework.

Choose by tools exposed, host support, auth and deployment.

# VS Code Copilot: setup and readiness

---

[github.com/pranavsaigandikota/MCPJAM](https://github.com/pranavsaigandikota/MCPJAM)

## Windows + Mac · Copilot account with agent access

- 1 Clone the repo; run the Windows or Mac setup.
- 2 Open the whole MCPJAM folder in VS Code.
- 3 Sign into Copilot; start the app for the lab.
- 4 Open `.vscode/mcp.json`; select your `.venv` Python.

**Copilot uses your account. Explicit CLI tests are the fallback.**

# Copilot demo: tempo + verified state

---

**“Set tempo to 120, then verify it.”**

- 1 Open Copilot agent chat; enable MCPJAM tools.
- 2 Inspect `set_tempo(bpm=120)` and its result.
- 3 Call `get_state`; check bpm is actually 120.



# MCP ACTIVITY · trace the call

---

**Host + MCP client**

VS Code Copilot · built-in MCP client

↓ Client ↔ server: MCP over stdio

**starter/mcp\_server\_sdk.py**

Tool definitions and the Python SDK

↓ Tool function → adapter: Python function call

**call\_daw()**

Backend adapter

↓ Adapter ↔ app: newline-delimited JSON over the local socket

**app.py**

Music Application · UI and audio

# Activity map: where we edit

---

| File                                      | Workshop step                                         |
|-------------------------------------------|-------------------------------------------------------|
| <code>.vscode/mcp.json</code>             | CONNECT · slide 21: interpreter + starter path        |
| <code>workshop/starter/</code>            | BUILD · slide 20: edit <code>mcp_server_sdk.py</code> |
| Copilot + <code>workshop_client.py</code> | TEST · slides 22–23: discover, call, verify           |
| <code>workshop/solutions/</code>          | REFERENCE · compare after your attempt                |

**MCP slides 14 + 16 trace the call. Music showcase: slide 27.**

## MCP ACTIVITY · understand the contract

---

```
@mcp.tool()
def set_tempo(bpm: int) -> str:
    """Set tempo from 40 to 240 BPM."""
```

### DECORATOR

Registers the tool

### TYPE ANNOTATION

Describes the input

### DOCSTRING

Describes its purpose

# A useful tool can act and observe

## 1 · READ STATE

Observe the starting value.

## 2 · PERFORM AN ACTION

Call a specific tool.

## 3 · READ STATE AGAIN

Verify the expected result.

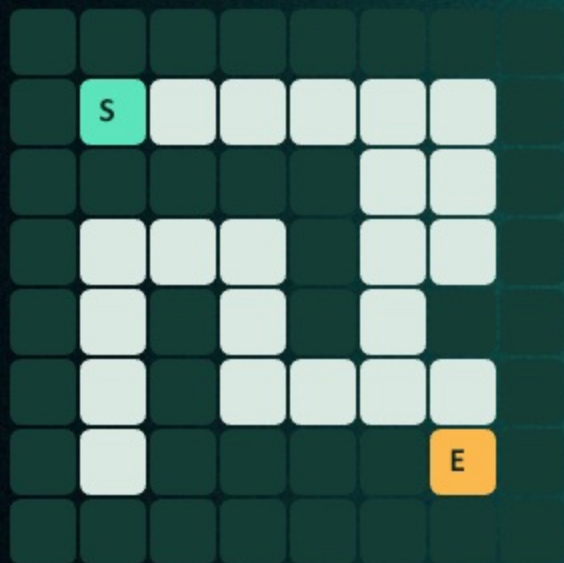
Trust “queued”  
or verify the actual state?



Choose the state check.

# Can an AI escape a maze using tools?

The model chooses a move. The MCP server exposes controls. The maze app applies it.



S = start    E = exit    Dark cells = walls

## EXPOSED MCP TOOLS

```
move_up()   move_down()
move_left() move_right()
get_maze_state()
```

## THE FEEDBACK LOOP

- 1 Read the maze + current position
- 2 Model selects a movement tool
- 3 App returns moved / blocked + position
- 4 Observe again; stop when at\_exit = true

## WORKSHOP CONNECTION

```
Move tool ↔ set_swing
get_maze_state ↔ get_state
Maze app ↔ app.py
```

**MCP is the interface. The model plans. The Application enforces valid moves.**

# Is this different from a “normal MCP”?

Same MCP protocol. Our custom server exposes music capabilities.

## **CODING AGENT · builds and tests**

- Shell, browser or API tools
- Can test an app without MCP
- Prompt alone adds no capabilities
- Verify with actual evidence

## **YOUR APP + MCP · reusable controls**

- Expose focused tools + schemas
- Reuse across compatible hosts
- Production: validate + authorize
- Read state to verify the result

**MCP is an integration choice. A direct API is enough for one fixed client.**

Use MCP when multiple compatible hosts need live, reusable app capabilities.

## MCP BUILD · six parts, one tool

EDIT: workshop/starter/mcp\_server\_sdk.py

1. Register `set_swing(amount: int)`.
2. Describe its purpose and 0–75 range.
3. Validate before calling the app.
4. `call_daw: cmd=set_swing, amount=input`.
5. Return the acknowledgement.
6. Read `get_state` to verify.

### SERVER

Reuse the supplied FastMCP instance.

### TOOL CONTRACT

Decorator + name + type + description.

### VALIDATION + ADAPTER

Check the range; send the app command.

### RESULT + TRANSPORT

Verify queued work. Keep stdio startup.

Add above `"if __name__"`. Save → restart the MCP server → verify discovery.

## Copilot connection: .vscode/mcp.json

Use the supplied JSON. The input prompt selects this clone's .venv Python.

```
"mcpjam": {  
  "type": "stdio",  
  "command": "${input:mcpjamPython}",  
  "args": [  
    "${workspaceFolder}/workshop/starter/mcp_server_sdk.py"  
  ]  
}
```

Python path: Windows → .venv/Scripts/python.exe

Mac → .venv/bin/python · enter an absolute path

MCP: List Servers → mcpjam → Start. After editing Python, restart the server.

# MCP TEST · discover, call, verify

---

- 1 Save your tool in workshop/starter/.
- 2 MCP: List Servers → mcpjam → Restart.
- 3 Enable tools; confirm set\_swing appears.
- 4 Ask for swing 35; verify with get\_state.

2 / 3 · PRIZE QUESTION · Hands up!

Should set\_swing(100) succeed?

A Yes    B No



**15 minutes.**  
**Test and adapt.**

**25–40 min · build, reconnect, test, then explain to a partner**

# MCP TEST · evidence of success

| Test            | Expected evidence                       |
|-----------------|-----------------------------------------|
| Discovery       | set_swing appears in available tools    |
| Amount 35       | Accepted and verified through get_state |
| Amount 0 and 75 | Boundary values accepted                |
| Amount 100      | Tool error; Application state unchanged |

3 / 3 · PRIZE QUESTION · Hands up!

Does “queued” prove the music changed?

A Yes    B No

When the valid call  
AND the rejected call  
both behave correctly.



Absolute verification.

# The security risks behind a tool call

## UNAUTHORIZED ACCESS

Tools or records exposed to a caller who should not have access.

## EXCESSIVE PERMISSIONS

A tool or backend account can do more than the task requires.

## INDIRECT INJECTION

Malicious instructions hidden in documents or tool results.

"Just give every tool admin access."



Range validation checks the value.

Authorization checks whether the caller may perform the action.

# Tool poisoning and rug pulls

---

## TOOL POISONING

Malicious instructions in a tool description try to manipulate the model or its workflow.

## RUG PULLS

A previously trusted tool changes its description or behavior after approval.

Review what you connect—and review what changes.



# Security controls for MCP tools

---

- ✓ Check access: identity, token audience, and permission for the action.
- ↓ Limit permissions: tools and backend accounts get only what they need.
- ! Treat external content as untrusted: documents and tool results can contain malicious instructions.
- ↻ Review code and updates: server sources, dependencies, and tool descriptions.

# Copilot music: prompt → tools → MP3

---

- 1 Music config: args → mcp\_server\_music.py.
- 2 In chat, type / and select compose\_music.
- 3 Description → web research\* → catalog → score.
- 4 Receive a 30-second MP3 path; open when ready.

\*Web access depends on enabled tools; a source URL can be fetched.  
Local stdio / remote HTTP · bounded calls · verify results · no autoplay.

# Enterprise: identity + production rules

---

## IDENTITY PROVIDER

Establishes identity  
Example: Entra ID

Identity / token issuance  
to the client

Tool request →

## OPTIONAL GATEWAY

Traffic policies and monitoring

MCP →

## MCP SERVER

Checks access to tools and data

Application call →

## BACKEND

Enforces Application business  
rules

Production: per-tool authorization · audit logs · safe retries · verified outcomes.

# Where can I use this at the hackathon?

---

## ONE USEFUL ACTION

Search inventory or create a draft ticket



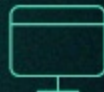
## ONE STATE CHECK

Read the item / ticket and verify the result



## ONE CLEAR BOUNDARY

Validate inputs; restrict access; approve risky writes



**Exit check: name your tool, input, permission, and the evidence that proves it worked.**